CS260 Reading Week 4

Question 1: Notes

11.1 Sorted Arrays

- sorted array -> search is O(log n) w/ binary search, good
- but insert/delete = O(n) bc gotta shift stuff to keep it sorted
- want something sorted AND fast insert/delete -> search trees (BSTs)

11.2 Supported Operations

- BST property: left subtree < node < right subtree, for every node
- ops (all O(height)):
  - search - compare to current node, go left or right
  - insert - search till it falls off tree, stick new leaf there
  - min/max - just go left/right all the way
  - predecessor/successor - in-order neighbor, use subtree if there, else go up thru parent pointers
  - delete - 3 cases: leaf (remove), 1 kid (splice out), 2 kids (swap w/ pred or succ, delete that instead)
  - select(i)/rank(key) - order stats, need subtree size stored at each node
- key thing: all ops are O(h) not O(log n) - only same as log n if tree is balanced
- height = shape of tree = depends totally on insertion order

11.3 Implementation (skim)

- node = key + left ptr + right ptr + maybe parent ptr
- balanced BSTs (red-black, AVL) keep O(log n) height no matter insertion order - later chapters
- can augment nodes w/ extra data (like subtree size) for stuff like select/rank, doesn't change big-O

Question 2: BST Height, n unique ints [1...n]

(a) max height?

Θ(n) - worst case height ~ n-1. bc if you keep inserting stuff that's always bigger (or always smaller) than what's there, every node just chains onto the last one, no branching. basically becomes a linked list.

(b) sequence for max height?

Sorted order - 1,2,3,...,n (or reverse, n,...,1). each new key is always the new max/min so it just attaches under the last node inserted. straight chain.

(c) min height?

Θ(log n), floor(log2 n) exactly.

sequence: insert median first, then median of each half, recursively (bisection order). eg n=7: insert 4, then 2 & 6, then 1,3,5,7.

why: splitting remaining keys in half each time keeps tree as complete binary tree instead of a chain. complete tree w/ n nodes has height floor(log2 n) - smallest possible, since each level can only hold ~2x the nodes of the level above it.